INFO 6150 / Prof. Fernando Augusto López Plascencia / Northeastern University

Week 11

Modern Apps; Business Modeling

In this lesson:

- The modern JS environment
  - The development stages and their tools
    - Package managers: npm
    - Configuring: scaffolders
    - Building: transpilers, bundlers, minifiers and packagers
    - Developing: live reloaders, linters, and test tools
  - Testing
    - Some testing types: unit, integration, end-to-end
  - Deployment
    - Choosing and configuring a host
    - Continuous Integration / Continuous Deployment (CI/CD)
  - Bonus: Project management with Scrum
- Use case modeling
- Business modeling using OOP principles

The modern JS environment

The modern environment

What if:

- you could run Javascript not just in your browser, but also in your command line or as a server program?
- you did not have to reload the page each time something changes?
- your tools told you when an instruction is wrong?
- you could easily download functions made by other people and use them in your app, instead of doing everything from scratch?
- you could configure all of the above easily?

These, and more, are the advantages of creating a project with a modern environment.

The modern environment

If you've already worked with React in your computer, you already used all of this: many pieces of software were working behind the scenes to make it happen.

This also is happening if you use an online development environment such as codesandbox or codeanywhere. In this case, everything is happening on a server somewhere on the internet. This is a valid option but it's not the cheapest one. It's better to understand what's going on and install it yourself in your machine, so that

- it can be always free,
- you have control over its configuration.

Node.js

- Node.js is a standalone interpreter for Javascript.
  - It allows running JS without the need of a browser.
  - This was the first product that allowed to do so.
- The most common usage of Node.js is as a web server.
  - In other words, it’s possible to write the “server” part of a web application.
- You can also use it to run general purpose JS scripts in your computer, much like Python or Java.

The Node.js environment

Traditionally, writing a script for Node means:

- You don't have a webpage, so you don't have the `window` or `document` globals.
- However, you have access to additional functions through the `process` global.
  - For example, reading and writing to local files

Node also contains lots of utilities specific to it. Many of them make it easier to write a server (a program that listens to network connections and responds to them) or do parallel processing.

The Node.js environment

We won’t go deeper into Node.js in this course but feel free to explore it at nodejs.org.

We will use it, however, to:

- Run a web server to host our app during development,
- Run our development tools.

At this point:

- If you haven’t installed Node in your computer, go to nodejs.org and install the latest version.

Npm and package management

Even if you don’t work on the server, you will want to use another, very part of the Node ecosystem: **npm** (Node Package Manager).

This is a system to share “packages”: JS code that is distributed through the internet and can be “imported” into your own program.

Why is package management important?

- To share and distribute code:
  - Packages are better to share and distribute code than writing hundreds of `<script>` tags in each page.
  - Package authors publish them through “registries”. One can search a registry to find packages that are useful.
- To manage dependencies:
  - Packages need other packages. This is because a package author may have used other packages in its creation.
  - Packages needed by other packages are called “dependencies”.
  - Managing dependencies is very complex (example: see `package-lock.json`).
  - An automated dependency management system is needed to do this correctly. The developer should not need to think about this.
- To keep dependencies up to date:
  - A package manager program can automatically check for new versions of dependencies, it can also install them.
  - This minimizes security risks and allows developers to stay current.

Many package styles: Module systems

Packages are, strictly speaking, “modules”. A module is a small piece of independent, reusable code.

```
const countlines =
(lines) => { return
lines.length; }

export default
countlines;
```

countlines

myapp.js

```
import countlines
from ‘countlines’;
…
const count =
countlines(arr);
```

Many package styles: Module systems

Node.js was the first system that allowed importing and exporting packages, so it came with its own definition: the CommonJS module system.

Instead of `import`, this system uses the `requires` keyword. It's a bit more flexible, but it's only for Node.

(There are other differences, but we don't need to go into that.)

Many package styles: Module systems

You may also see the keyword `require` in other style of modules called AMD modules (for Asynchronous Module Definition). These are different in that they were invented for the browser, rather than Node. They require an external `<script>` to be used.

Many package styles: Module systems

Nowadays, it's also possible to use modules natively in the browser thanks to the "official" module system for Javascript, the ECMAScript modules. These modules run both in Node.js and in the browser.

When we use `import` and `export`, we're following a standard defined by ECMAScript.

Thanks to ECMAScript, now it's possible to define modules out of normal files that the browser can read directly, without the need for other programs. More on that here. This can be useful for smaller apps where a complex architecture is not needed.

Many package styles: Module systems

In short, there are three main module systems:

- CommonJS
- AMD
- ECMAScript modules - we will use these

Package managers

We will use npm, the package manager included with Node, since that's the standard.

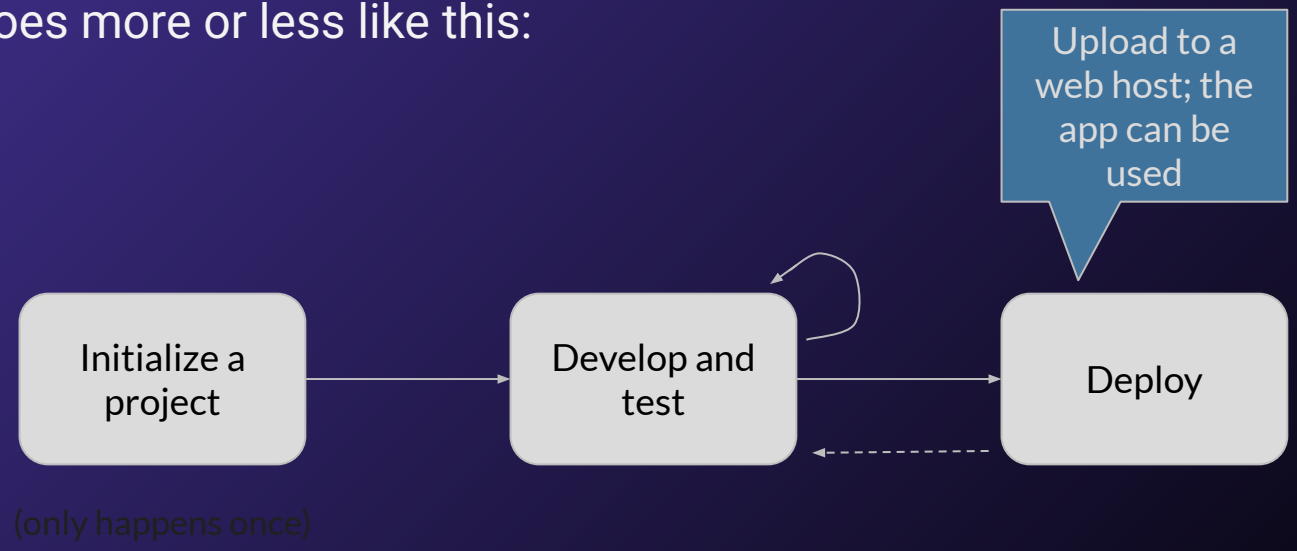
However, there are alternatives, such as:

- Yarn, created to be much faster than npm, and defines its own files;
- Pnpm, as fast as Yarn but with files compatible with npm.

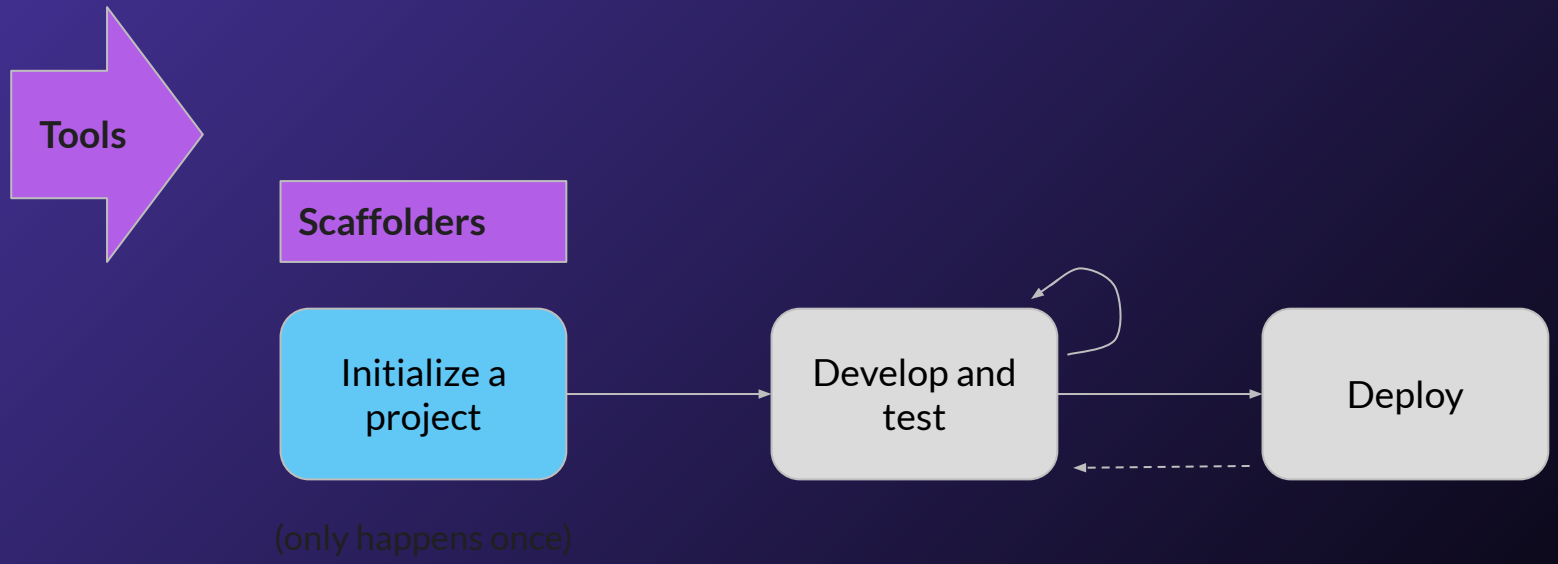
You will be okay with using npm unless you have very specific needs or are having trouble with compilation speed. (More on that later.)

The modern JS development process

...goes more or less like this:



Initializing a project



Initializing a project

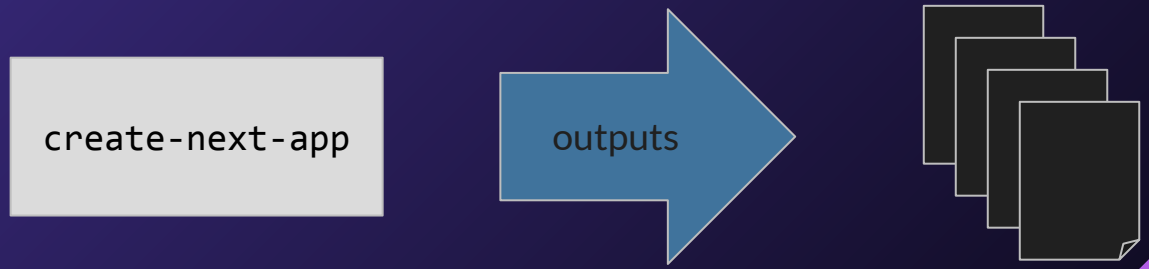
- Choosing a technology
- Scaffolding (creating the project)

Initializing a project

- Choosing a technology
  - “Vanilla” Javascript, React, Vue, etc.
  - Define an architecture
    - Meaning: what goes in where
    - Choose what other packages you will use
- Scaffolding (creating the project)

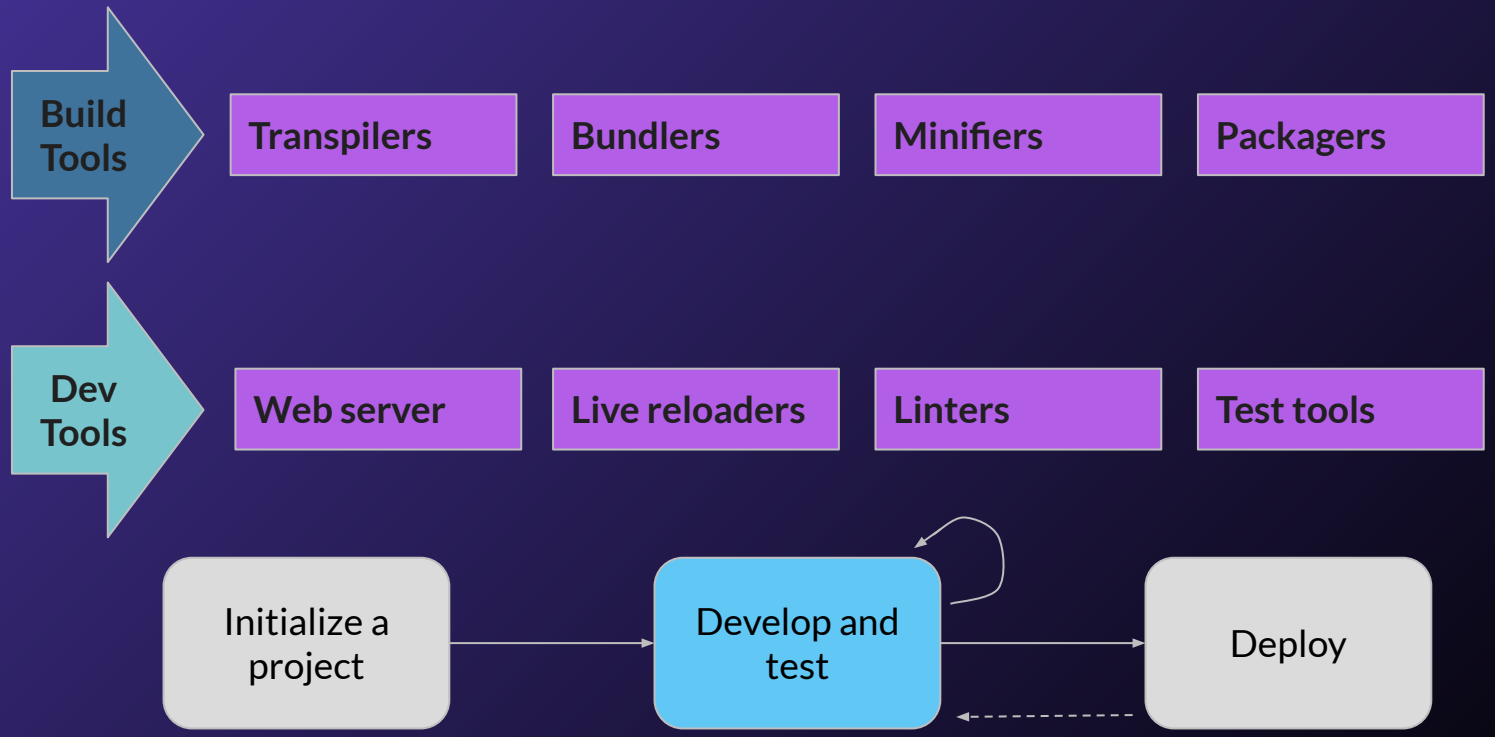
Initializing a project

- Choosing a technology
- Scaffolding (creating the project)
  - “Scaffolders”: scripts to set up a project; they install packages and development tools
    - Examples: `create-next-app`, Yeoman
    - Come with many possible configuration options to make your life easier
  - Or you can install everything manually



Your project’s files and configuration

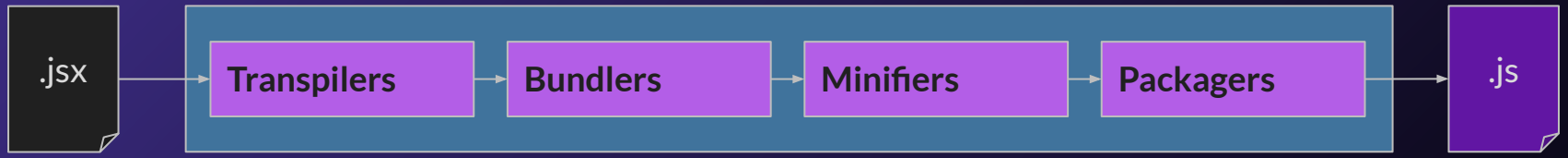
Developing and testing



Developing and testing

- This is what you'll be doing most of the time.
- How your tools work together: Let's talk about the build tools first.
  - The build is the process to convert your React files into Javascript that the browsers can use.

The build process

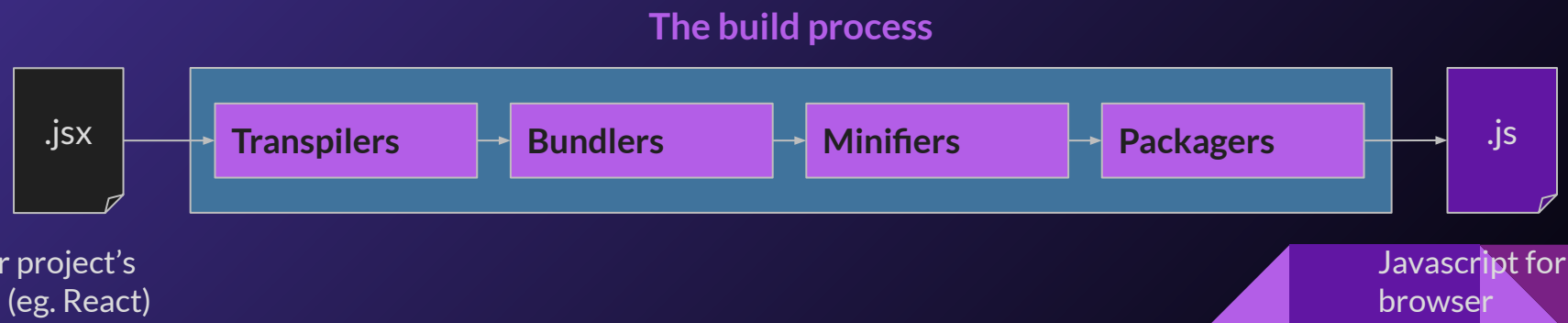


Your project's files (eg. React)

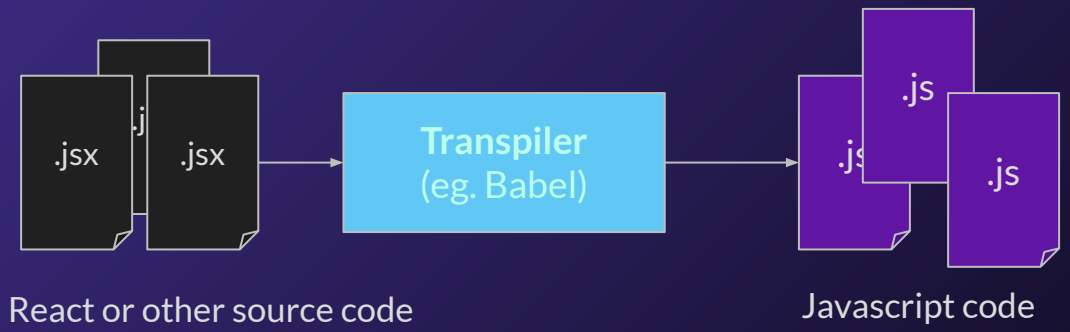
Javascript for the browser

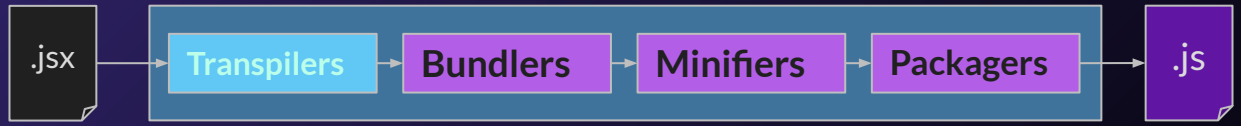
The build process

- This is the process that transforms your React code (or other source code) into the Javascript code that the browser (or Node) will actually execute.
- While this happens automatically, there are many parts to it:



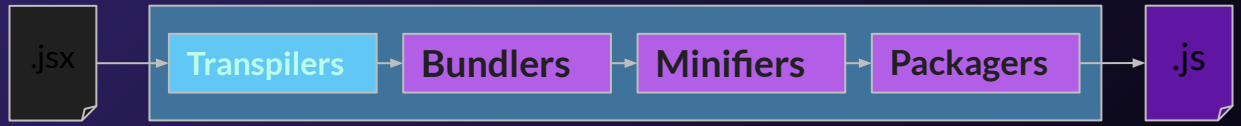
The build process: Transpilers



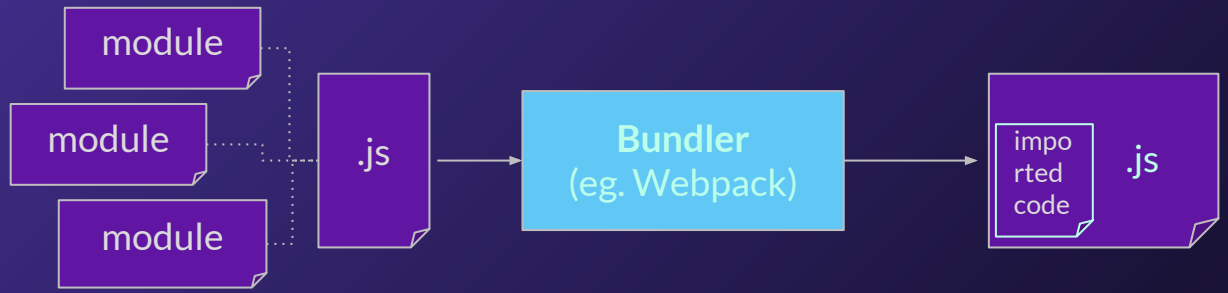


The build process: Transpilers

- The React code cannot be read by a browser. It needs to be converted into standard Javascript first.
  - There are other technologies that need this: for example, Typescript. There's also Sass, which needs to be converted to standard CSS.
- The process to convert one source code into another is called "transpilation".
  - We said earlier that React needs to be "compiled". This is not really exact.
  - "Compilation" actually means: converting source code into machine code
- Transpilers can read code from these different source codes and do the transpilation.
- The most popular is probably Babel.

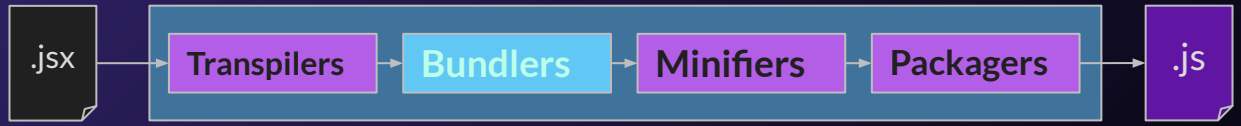


The build process: Bundlers



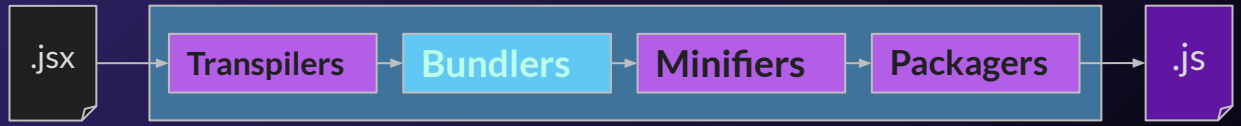
Javascript with dependencies

“Bundle”:
Big Javascript file including code copied from modules (“resolved” dependencies)



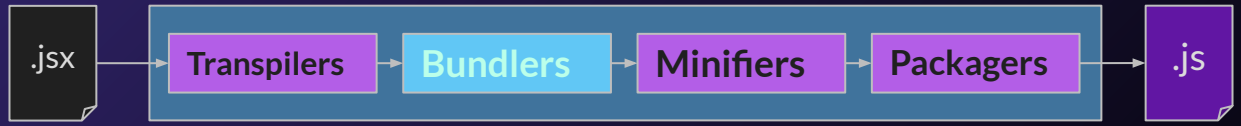
The build process: Bundlers

- The Javascript code still has a lot of `import` (or maybe `require`) statements in it, because we are using code from other packages (modules) in it.
- Browsers used to not be able to interpret this.
  - Things are changing now that native modules exist.
- The bundler's job is to find the packages used by your code (the "dependencies") and putting the relevant code together in a single, large file.
  - In other words, it "solves" the dependencies.
  - This big javascript file is called the "bundle".
    - Making the bundle as small as possible is a common theme for website optimization.



The build process: Bundlers

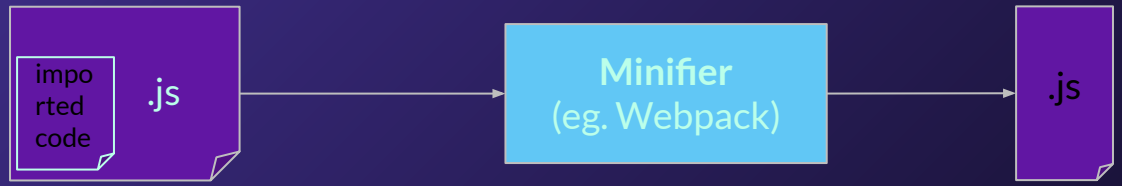
- The choice of a bundler is very important. It is often the “central piece” of the whole configuration.
  - The bundlers are very powerful, so they’re often used to direct the whole build process. They can also manage configuration for the other tools.
- The first and most common bundler is webpack.
  - Webpack is very powerful, but very complex.
- Many alternatives exist, such as Rollup, Gulp, Browserify, Parcel, Vite, Brunch…



The build process: Minifiers

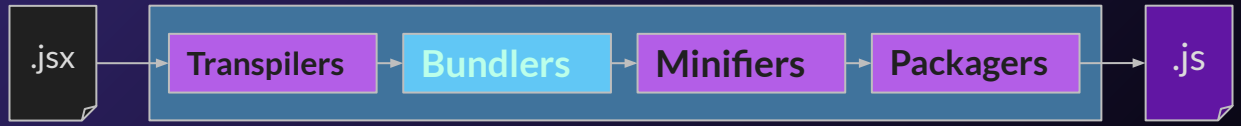
```
const times = 100;
for (let counter = 0; counter <=
times; counter = counter + 1) {
```

```
const t=100;for(let
c=0;c<=t;c++){
```



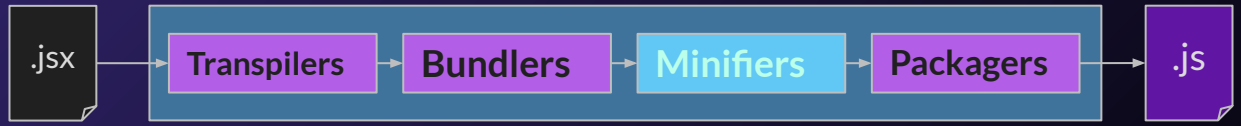
"Bundle":
Big Javascript file including code copied from modules ("resolved" dependencies)

Minified, obfuscated bundle

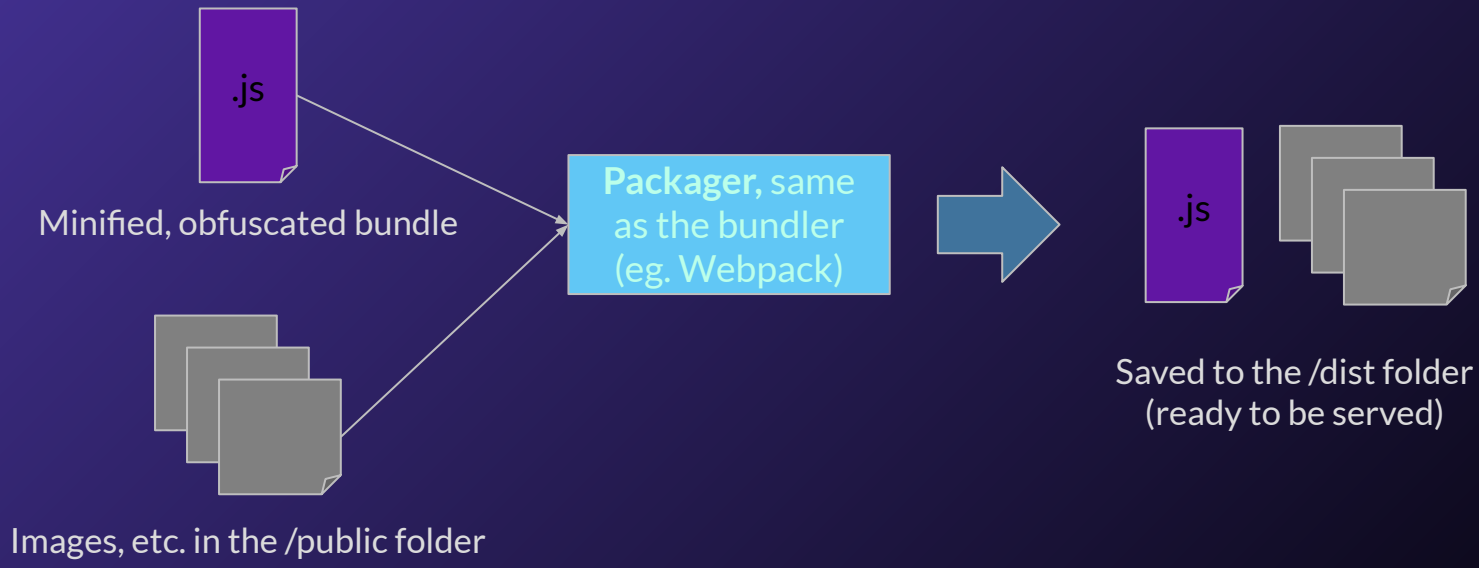


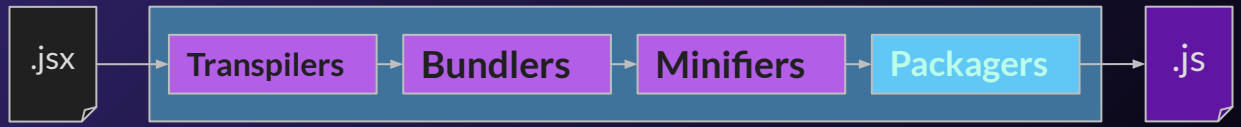
The build process: Minifiers

- At this point the Javascript file can be read by any browser because it includes all code it needs from its modules… but it became very, very big.
  - We want to save bandwidth at all times, so files (including Javascript) should be small.
- Additionally, the file could be read by anyone using the website. This means a person with bad intentions could understand how to trick the website. This would be bad.
- Minifiers help with these two things at the same time: they compress the code to make the file smaller, while also making it very difficult to read.
  - Try viewing the contents of a JS file loaded by a commercial website. It will probably be minified.
- Uglify.js and Minify are two examples of minifiers.



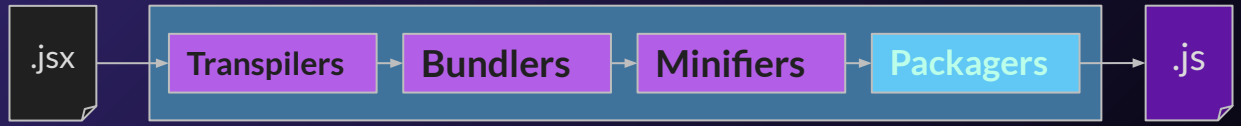
The build process: Packagers



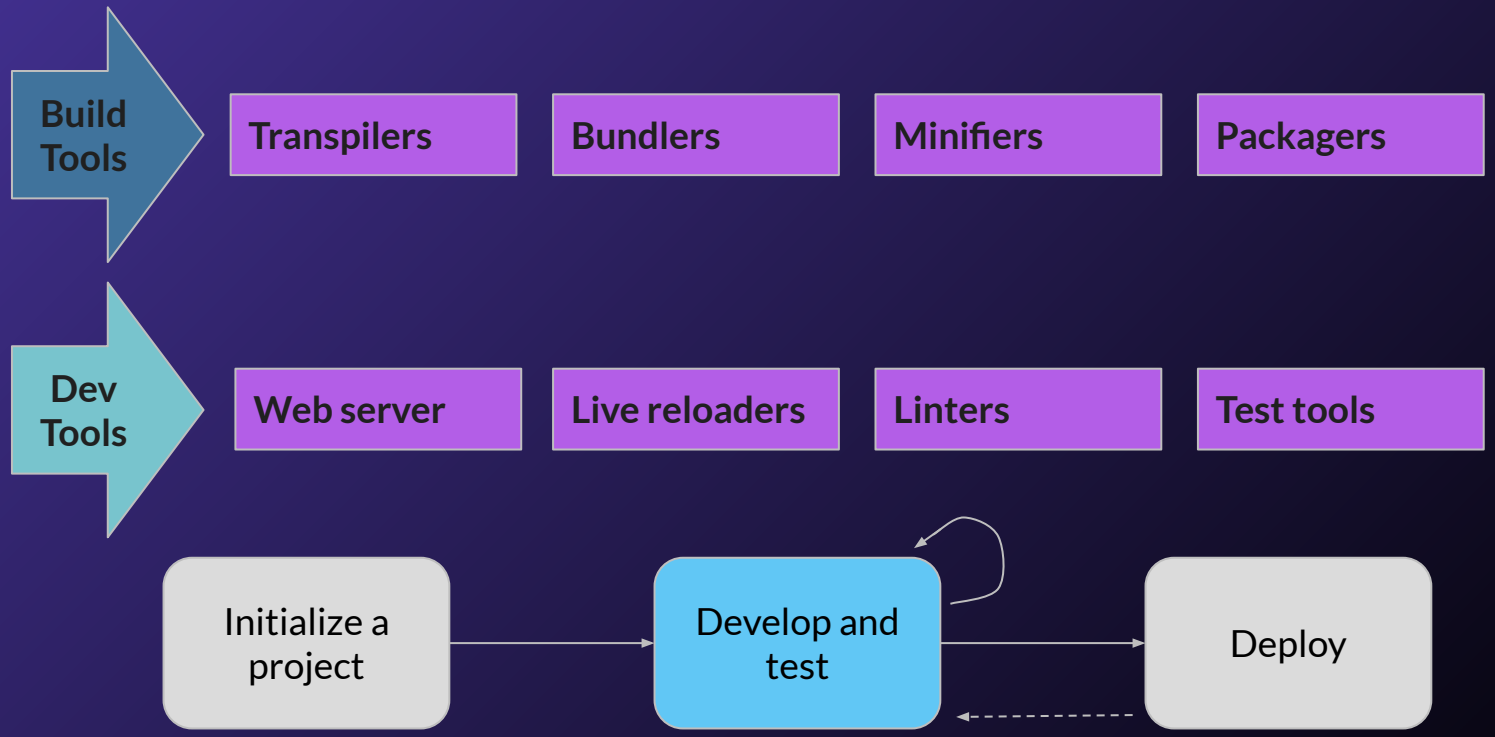


The build process: Packagers

- Now we have the file ready to be served… or almost! We need to put it in the right place within our project.
  - For React, by default, source files live in a directory (such as `/src`) and resources are in another (such as `/public`).
  - We need to put the resources and the bundle in a new directory (such as `/dist`). The contents of this directory are what actually gets uploaded to our server.
- Copying the bundle and related files into the distribution folder is the job of the packager.
- Nowadays, the bundlers are also packagers, so there are no separate products here.



Developing and testing



Development tools

Let's talk now about the tools that you will use to help yourself while coding.

- The web server: In this case we're talking about servers for development: the ones running in your own machine on http://localhost:3000 for example.
  - Usually lightweight, running in Node.js.
  - When your project runs (`npm start`) it starts the web server
  - Having a web server allows for automatic integration with the other tools.

Note: web servers for production are their own, very complex software; examples are Apache, nginx and IIS. You don't need one of these to develop; they live in the production servers (in the internet).

Web server | Live reloaders | Linters | Test tools

Development tools

Let's talk now about the tools that you will use to help yourself while coding.

- Live reloaders (or hot reloaders): This allows you to see changes in your browser as soon as you change your source files.
- It's composed of two main parts:
  - A process (a running program) in your computer notices when you save a file, then runs the whole build process for the project.
  - A Javascript "monitor" script is attached to the webpage served in your localhost. This loads the new bundles as soon as they're ready.
    - If there are errors, this monitor also will show the errors for you.
    - Another program, a "mapper", gives you links from the error to the exact part of your source (eg. React) code where the error happened.
- Examples: nodemon (comes with Node.js) and the one included with the Webpack bundler.

Web server | Live reloaders | Linters | Test tools

Development tools

Let's talk now about the tools that you will use to help yourself while coding.

- Linters: Will warn you in case you break a style rule.
  - This is important to keep your code clean and also to detect potential bugs.
  - Style rules are configurable. You can choose to use style rules from the internet or create yours.
  - They can be integrated to your editor (eg. VS Code) or run separately in your machine
  - Linting can also be run as part of the build process.
- The most popular nowadays is probably eslint.

Web server | Live reloaders | Linters | Test tools

Development tools

Let's talk now about the tools that you will use to help yourself while coding.

- Test tools: This actually deserves its own section (coming up next). They run tests to make sure your code does not break.
  - Tests can also run as part of your build process (and probably should!).
  - Test tools are only ways to run tests. You must write the tests yourself.

Web server | Live reloaders | Linters | Test tools

Testing

Testing is such a big topic that it deserves its own course! For now, let’s cover some basics.

We test to:

- Make sure the code we write does what it’s supposed to do;
- Ensure that new code does not break how older code works;
- Describe the code (as a form of documentation).

We need to do this at many levels, both testing each small piece of code and also how these pieces work together.

Testing

Non-automated testing: These are tests that you do yourself, without any tooling.

- Read your program's requirements again.
- Check that each requirement is being met.
  - Find edge cases and make sure your program works well with them.

You can write test plans to describe each test.

In the simplest of test plans, each test is described as follows:

- What the user needs to do,
- What the program should do in response.

When running the test, you also write what the program actually did, specially if it failed.

Testing

Non-automated testing: These are tests that you do yourself, without any tooling.

- Read your program's requirements again.
- Check that each requirement is being met.
  - Find edge cases and make sure your program works well with

You can write test plans to describe each test.

In the simplest of test plans, each test is described as follow

- What the user needs to do,
- What the program should do in response.

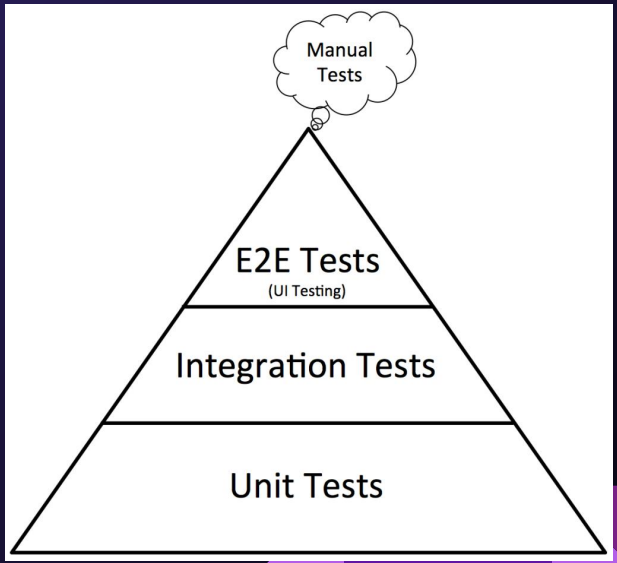
When running the test, you also write what the program actually did, specially if it failed.

This is super tiring, and does not scale well!

Automated testing

We will talk about the three main types.

- Unit tests: They test small units of code: a function, class, or component by themselves
  - Most of your tests should be unit tests
- Integration tests: Test how units of code work together
  - Have less of these, but enough
- End-to-end tests: Test how users work in a whole program
  - These should be the least amount



Source: https://www.ministryoftesting.com/articles/the-mobile-test-pyramid

Automated testing

We will talk about the three main types.

- Unit tests: They test small units of code: a function, class, or component.
  - The “unit” is tested in isolation: other parts of the code are simulated (“mocked”) so that we can control their inputs and outputs.
- Very simple and easy to do if the functions and components are pure.
- There are many frameworks for JS unit testing. Some of the most popular ones are Jest, Jasmine and Mocha.

Automated testing

We will talk about the three main types.

- Unit tests: an example
  - Consider this function that returns the absolute value of a number:

absolute.js

```
export const absolute = (num) => {
    return num >= 0 ? num : -num;
}
```

Automated testing

We will talk about the three main types.

- Unit tests: an example
  - In a file in the same directory, these could be the unit tests (written with Jest):

absolute.spec.js

```
import jest from 'jest';
import { absolute } from './absolute.js';

describe('absolute', () => {
    it('returns the same number if positive', () =>
{
        const result = absolute(3);
        expect(result).toBe(3);
    });
```

```
    it('returns the positive if parameter is negative',
() => {
        const result = absolute(-5);
        expect(result).toBe(5);
    });

    it('returns NaN if the parameter is not numerical',
() => {
        const result = absolute('abc');
        expect(Number.isNaN(result)).toBeTruthy();
    });
});
```

Automated testing

We will talk about the three main types.

- Unit tests: an example
  - In a file in the same directory, these could be the unit tests (written with Jest):

absolute.spec.js

```
import jest [illegible] 'jest';
import { al[illegible]js';

describe('[illegible]
    it('re[illegible]sitive', () =>
{
        const result = absolute(3);
        expect(result).toBe(3);
    });
```

“.spec.js” is a convention; means there’s a test in there

```
    it('returns the positive if parameter is negative',
() => {
        const result = absolute(-5);
        expect(result).toBe(5);
    });

    it('returns NaN if the parameter is not numerical',
() => {
        const result = absolute('abc');
        expect(Number.isNaN(result)).toBeTruthy();
    });
});
```

Automated testing

We will talk about the three main types.

- Unit tests: an example
  - In a file in the same directory, these could be the unit tests (written with Jest):

absolute.spec.js

```
import jest from 'jest';
import { absolute } from './absolute.js';

describe('absolute', () => {
    it('returns the same number if positive', () =>
{
        const result = a[illegible]e(3);
        expect(resul[illegible]
    });
```

Define each test by stating what you expect from them

```
    it('returns the positive if parameter is negative',
() => {
        const result = absolute(-5);
        expect(result).toBe(5);
    });

    it('returns NaN if the parameter is not numerical',
() => {
        const result = absolute('abc');
        expect(Number.isNaN(result)).toBeTruthy();
    });
});
```

Automated testing

We will talk about the three main types.

nple

ne directory, these could be the unit tests (written with Jest):

Tests check the “golden path” (eg. what it’s supposed to happen…)

…and also edge / error cases.

```
ab
i
import { absolute  ./absolu

describe('absolute', () => {
    it('returns the same number if positive', () =>
{
        const result = absolute(3);
        expect(result).toBe(3);
    });
```

```
    it('returns the positive if parameter is negative',
=> {
        const result = absolute(-5);
        expect(result).toBe(5);
    });

    it('returns NaN if the parameter is not numerical',
() => {
        const result = absolute('abc');
        expect(Number.isNaN(result)).toBeTruthy();
    });
});
```

Automated testing

We will talk about the three main types.

- Unit tests: an example
  - In a file in the same directory, these could be the unit tests (written with Jest):

absolute.spec.js

```
import jest from 'jest';
import { absolute } from './absolu

describe('absolute', () => {
    it('returns the same number if positive', () =>
{
        const result = absolute(3);
        expect(result).toBe(3);
    });
```

This test will fail. Writing the test makes us notice that. We can go fix it now.

```
    it('returns the positive if parameter is negative',
=> {
        const result = absolute(-5);
        expect(result).toBe(5);
    });

    it('returns NaN if the parameter is not numerical',
() => {
        const result = absolute('abc');
        expect(Number.isNaN(result)).toBeTruthy();
    });
});
```

Automated testing

We will talk about the three main types.

- Unit tests: an example
  - In our command file, we can run the test command manually (for example `npm run test`).
  - This is an example of how it looks like.
  - LLMs are great at writing unit tests.

```
$ npm run test

 PASS  src/tests/Counter.test.js (15.279 s)
  √ increments counter (204 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        31.064 s
Ran all test suites related to changed files.
```

Source: https://www.browserstack.com/guide/unit-testing-of-react-apps-using-jest

Automated testing

We will talk about the three main types.

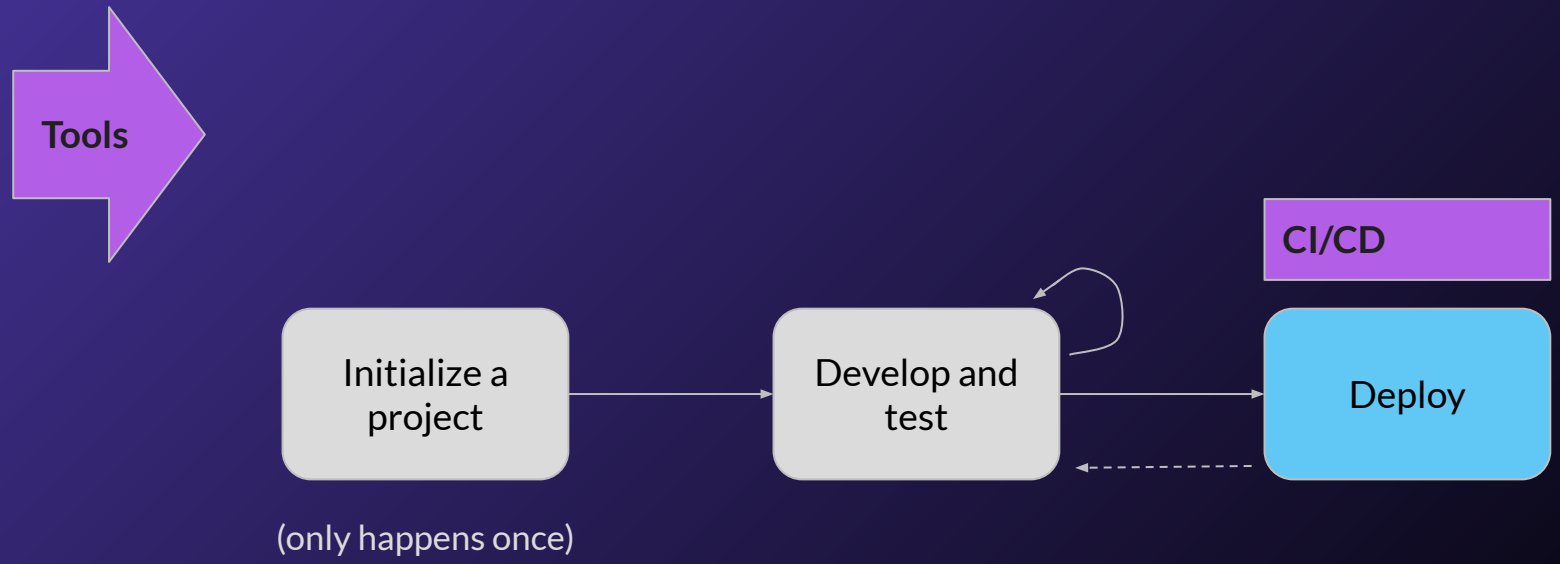
- Integration tests: Similar to unit tests, but do not mock some (or most) inputs. The idea is to see how your component / function / class works together with them.
  - Exception: you should always mock API calls!
- The same software for unit testing can be used for integration testing.
- You should be picking small groups of code to test together!
  - For example, a React component and their children, but not a whole page.
- When testing components, you can simulate eg. clicks, typing text, etc. You would use the DOM for this.

Automated testing

We will talk about the three main types.

- End-to-end (e2e) testing: The idea is to test use cases: the ways in which your application can be used by the users. Use cases come directly from your requirements.
  - When you are testing manually, you are doing non-automated end-to-end testing.
- Software for e2e testing is complex and heavy. Includes:
  - “Headless browsers” that can render a page in memory, instead of in a browser
  - Automated “actors” that interact with the browsers. You can program them with sequences of clicks and types, etc. but as if they happened in a browser (eg. not through the DOM).
- Examples of software include Cypress, Playwright, and Selenium.
- LLMs do not excel at writing e2e tests but there is some success, for example, by applying agentic workflows (example use case).

Deploying a project



Deploying a project

“Deployment” means sending the finished files to a “host” (web server + disk space + other services). The host is connected to the web and configured so that other people can use the app.

This mainly concerns two things:

- Choosing and configuring your host
- Configuring continuous integration / deployment

Deploying a project: Choosing a host

There are many commercial hosts. You will have to choose a host that:

- Supports the infrastructure for your project - in this case, they have to support Node.
  - Ideally, they directly support the framework you're using.
  - Some hosts specialize in certain architectures, such as React with Next.js for example, but most are flexible.
- Has a good plan with fair prices and good support.
  - Consider your costs!
  - Hosts usually have packages that include some disk space, bandwidth, etc. per week / month / year.
- Can meet your app's requirements.
  - For example: if you expect that your app will suddenly be used by lots of people, can the host scale? How fast are their services, and will that be enough for your app? Etc.

Deploying a project: CI/CD

While it's totally okay for very small projects to just upload your files (eg. the content of your `/dist` directory) to your host's server using ftp or similar, you may prefer a continuous integration option.

Continuous Integration / Continuous Deployment (CI/CD) are the technologies that automate the process of

- Detecting when you have new files in your repository or other place,
- Triggering the build process for your project, including any tests, and
- Copying the generated files to the host.

For bigger projects, CI/CD is essential. Configuring and maintaining servers and CI/CD is usually the domain of DevOps professionals.

Deploying a project: CI/CD

Example: The instructor's website, https://fernandolopez.dev, is currently hosted on Netlify under the free plan.

Netlify's CI/CD integrates with GitHub and detects merges to the main branch of the website's repository.

On each merge, the CI/CD triggers a new "build".

Builds | sgenius | Netlify

app.netlify.com/teams/sgenius/builds

Google Keep | The Sunday Mass |... | Read JavaScript Allo... | ファイル - SkyDrive | Learn to Code by D... | Collective Health | Northeastern | All Bookmarks

Fernando Augusto López Plascencia's team

Search Netlify...

News | Support

- Team overview
- Sites
- Builds
- Integrations
- Domains
- Members
- Audit log
- Security Scorecard
- Billing
- Team settings
- Visual editor dashboard

Upgrade

Team builds status

Starter

Total data from this usage period (Mar 17 to Apr 17), updated hourly.

Concurrent builds	Total build minutes used	Minutes used last period
0/1	0/300	0

Manage build capacity

- Builds
- Usage & insights

Builds

www.fernandolopez.dev: Production: main@367ea47 Completed
By sgenius: Merge pull request #3 from sgenius/mosaic-better-zoom
Apr 12, 2023 at 8:51 PM
Deployed in 1m 46s

www.fernandolopez.dev: Deploy Preview #3: mosaic-better-zoom@8933be0 Completed
By sgenius: badly zoom on mobile
Apr 12, 2023 at 8:51 PM
Deployed in 1m 40s

www.fernandolopez.dev: Deploy Preview #46: dependabot/npm_and_yarn/tar-4.4....@dec16c0 Completed
By dependabot[bot]: Bump tar from 4.4.13 to 4.4.19
Aug 31, 2021 at 10:00 PM
Deployed in 2m 41s

www.fernandolopez.dev: Deploy Preview #45:
Aug 10, 2021 at 5:39 PM

Build details | Builds | sgenius |

app.netlify.com/teams/sgenius/builds/64377c56e163e20008d55c4b

Google Keep | The Sunday Mass |... | Read JavaScript Allo... | ファイル - SkyDrive | Learn to Code by D... | Collective Health | Northeastern | All Bookmarks

Fernando Augusto López Plascencia's team

Search Netlify... | News | Support

- Team overview
- Sites
- Builds
- Integrations
- Domains
- Members
- Audit log
- Security Scorecard
- Billing
- Team settings
- Visual editor dashboard

Upgrade

peaceful-haibt-dc3f6d is a Next.js site!

Netlify auto-detected Next.js and will use the **Next.js Runtime** to build and deploy your site.

Read more about deploying with Next.js on Netlify

Deploy summary

275 new files uploaded
258 generated pages and 17 assets changed.

647 redirect rules processed
All redirect rules deployed without errors.

1 header rule processed
All header rules deployed without errors.

No functions deployed
This deploy did not include any functions. Learn more about Netlify Functions

No edge functions deployed
This deploy did not include any edge functions. Learn more about Edge Functions

Built using the Next.js Runtime
Netlify auto-detected Next.js and used the Next.js Runtime to build and deploy your site. Learn more about deploying with Next.js on Netlify

Build time: 1m 34s. Total deploy time: 1m 46s
Build started at 8:53:38 PM and ended at 8:55:12 PM. Learn more about build minutes

Build details | Builds | sgenius |

app.netlify.com/teams/sgenius/builds/64377c56e163e20008d55c4b

Google Keep | The Sunday Mass |... | Read JavaScript Allo... | ファイル - SkyDrive | Learn to Code by D... | Collective Health | Northeastern | All Bookmarks

Fernando Augusto López Plascencia's team

Search Netlify... | News | Support

- Team overview
- Sites
- Builds
- Integrations
- Domains
- Members
- Audit log
- Security Scorecard
- Billing
- Team settings
- Visual editor dashboard

Upgrade

Deploy log

Preview | Maximize log

Initializing	Complete
Building	Complete
Deploying	Complete
Cleanup	Complete
Post-processing	Complete

Deploy file browser

Start browsing through this deploy's 1,219 files.

Browse files

An aside: project management with Scrum

Project management is its own topic and deserves a course of its own. Suffice it to say that it comes with special tools and techniques as well.

In the 80s and before, software was mostly done through the waterfall methodology.

In this methodology, each part of the process is done once and handed to the next team. It's just a series of steps, so it's simple,

However, context was lost often from one team to the other. The whole project was delivered at once, leaving little room for corrections.

Waterfall can still be okay for smaller projects, though!

An aside: project management with Scrum

However, things have changed. The most popular way to manage software projects nowadays is the agile methodology

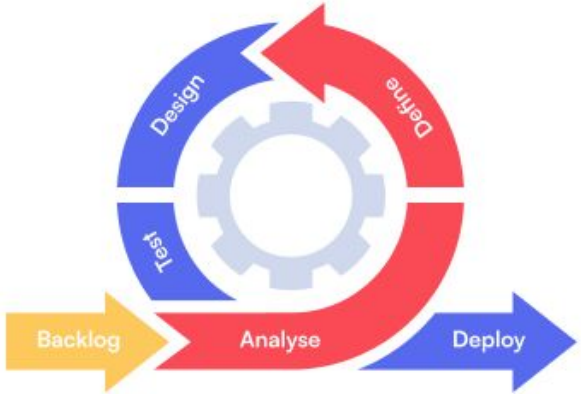
Agile says: Deliver the project little by little. Work in cycles. In each cycle, have a small objective. For each cycle, follow each step of the development process as needed. Agile is based on this manifesto.

Agile emphasizes collaboration and communication between teams and with the client. Feedback is gathered fast and corrections can be made. The user is happy because the product keeps improving and bugs are fixed. With Agile, it's also easier to keep a project well managed (meaning: in time and within budget).

The downside is that Agile takes some effort and commitment from the team to implement.

An aside: project management with Scrum

Waterfall methodology



Agile methodology

Source: https://testsigma.com/blog/difference-between-agile-and-waterfall/

An aside: project management with Scrum

There are many techniques to apply the agile philosophy, such as Kanban and Scrum.

Scrum is possibly the most common project management methodology found in software companies nowadays. It means (among other things):

- Teams break down projects, for example, in “epics” (big objectives) and “tasks” (small)
- Cycles are called “sprints” and have a fixed duration, such as one or two weeks.
- Before each sprint, tasks are measured depending on the effort they take. Each person gets more or less the same workload.
    - Each sprint has goals, and everyone has a clear idea of their goal
- After the sprint, a “retrospective” is done to check if everyone could finish the work.
- Often there is a “scrum master” who takes care of organizing all of this

Teams choose to adopt as much or as little of Scrum as they need.

Scrum is so popular that there is software for it. Jira and ClickUp are two examples.

Use Case Modeling

What is Use Case Modeling for?

At this point you know how to program a very simple app. But what happens if the app is not simple… like your final project?

When requirements are complex, we need a way to make sure:

- All requirements are captured,
- The requirements are still easy to understand,
- The team has a standard way to refer to each requirement.

We will study an industry standard: use case modeling.

What is a use case? What is a user?

A user is really a user *type*.

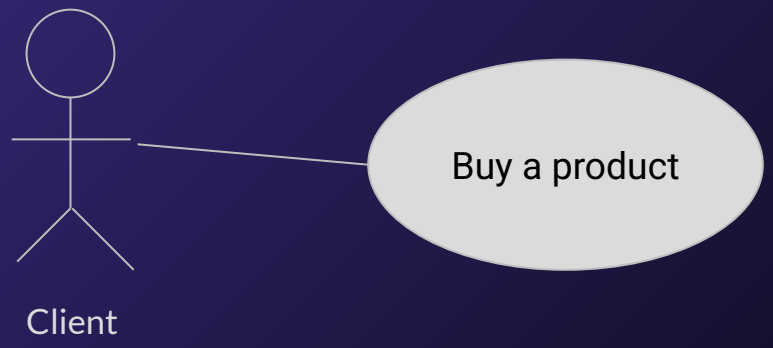
- Not “Mr. Bean”, “Ms. Jill” but rather “Client”, ‘Manager”, “Administrator”...
- A good user is one you can describe clearly. They have specific responsibilities
- It is possible for the same *person* to be different kinds of users in the same system
    - For example: if Mr. Green, the manager of a store, also buys there, he is both manager *and* client

Manager

What is a use case? What is a user?

A use case is a specific thing that a user will do in a system, from start to end.

- For example, for a system such as Amazon.com: “the client buys a product”



Describing use cases

Each use case is well defined when it has a sequence of actions.

Describe that from the start of the end of the interaction of the user with the system.

You should describe this as a conversation:

- The user inputs
- The system responds

The list should describe any decisions that happen as well.

You're writing the algorithm of this operation for when everything goes well: this is often called "the golden path"

Describe a use case: example

The Client buys a product:

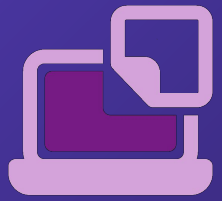
1. The client searches for a product. The system shows a results page.
2. The client clicks on the desired product. The system shows a page for this product, with a description, photos, and a “buy” button.
3. The client clicks the “buy” button. The system adds the product to the shopping cart.
4. The client clicks on the “shopping cart” button. The system shows the shopping cart page, with the list of products to buy and a “pay” button.
5. The client clicks on “pay”. The system shows a page asking for credit card information.
6. The client enters credit card information and clicks “finish”. The system contacts the bank to get funds from the credit card and, once this is successful, shows a “thank you for buying” page with information about the buy.

Use Case Modeling

You will try to identify:

- What gets done: the use cases
- Who does it: the users

Then, you will create a diagram with this, and optionally text descriptions.



Created by HideMaru
from Noun Project

Getting requirements into use cases

Imagine you were hired to make a system for Starbeaks, a coffee place.

Ask the manager, Mr. Bean, what they need in the system. (It will be a system for coffee orders and also for tracking employee shifts.)

Try to identify:

- Users: who needs to use the system?
  - Remember: This is not “Mr. Bean”, “Ms. Jill” but rather “Client”, ‘Manager”, etc.
- Use cases: what do they want to do in the system?

Tip: a system will (at least initially) emulate what users do in real life.

Icon by HideMaru for Noun Project

Getting requirements into use cases

Users:

- Cashier: Gets the orders in the system
- Barista: makes the coffee orders from the system, gives them to the client
- Manager: checks the orders of the day, time shifts, etc.

Getting requirements into use cases

Use cases:

- Cashier:
  - Enter order
  - Clock in
  - Clock out
- Barista:
  - Clock in
  - Clock out
  - Mark order as completed
- Manager:
  - Check orders of the day
  - Add / delete / edit shifts for next week

Getting requirements into use cases

Use cases:

- Cashier:
  - Enter order
  - Clock in
  - Clock out
- Barista:
  - Clock in
  - Clock out
  - Mark order as completed
- Manager:
  - Check orders of the day
  - Add / delete / edit shifts for next week - this is a CRUD, we'll talk about this soon

Getting requirements into use cases

Make the corresponding diagram. Then, describe at least one use case, from start to end.

Business Modeling

Business modeling? Why?

The use cases tell you what the system needs to do and how. The next step is to figure out the shape of the data you will use and how it relates to each other. This is what business modeling does.

Imagine you are making a simple app to manage books in a library. You are just making a CRUD for books. Easy, right?

Photo by Rumeysa Demir:
https://www.pexels.com/photo/organized-library-shelves-with-books-30744505/

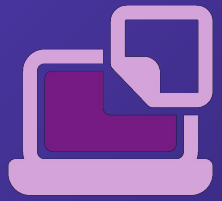
Wait, what is a CRUD app?

CRUD means:

- Create
- Read
- Update
- Delete

which are the operations of a very basic app.

So, “a CRUD for books” means a simple app that creates, reads, updates, and deletes books.



Created by HideMaru
from Noun Project

Business modeling? Why?

But reality is much more complex than that. What are other use cases for the library app?

- Discover the users, then the use cases.
- Make a list of use cases, ordered by user.

Icon by HideMaru for Noun Project

Business modeling? Why?

The users in your library will need to do all sorts of stuff. You may have something like this:

- Library member:
  - Check out books
  - Return books
  - Pay fees for overdue books
  - Find where in the library is a specific book:
    - For a certain author
    - For a theme
- Library employee:
  - Knowing if a library member has overdue books
  - Send reminders to members to return their overdue books
  - Login and logout of the system

Etc.

Business modeling? Why?

Suddenly, we no longer have just a CRUD for books! Our use cases are much more complex than that!

Business modeling? Why?

To understand how to design such a complex system, we need to first understand and describe the things (“entities”) we work with and how they relate to each other.

Business modeling? Why?

To understand how to design such a complex system, we need to first understand and describe the things (“entities”) we work with and how they relate to each other.

- In the library:
  - We have books with authors and themes. They are stored in library places identified by eg. aisle numbers. We can assign a code to each book to locate it in each aisle.

Business modeling? Why?

To understand how to design such a complex system, we need to first understand and describe the things (“entities”) we work with and how they relate to each other.

- In the library:
  - We have books with authors and themes. They are stored in library places identified by eg. aisle numbers. We can assign a code to each book to locate it in each aisle.
  - We have many copies of the same book. So, we have book *titles*, with a number of *copies* each.

Business modeling? Why?

To understand how to design such a complex system, we need to first understand and describe the things (“entities”) we work with and how they relate to each other.

- In the library:
  - We have books with authors and themes. They are stored in library places identified by eg. aisle numbers. We can assign a code to each book to locate it in each aisle.
  - We have many copies of the same book. So, we have book *titles*, with a number of *copies* each.
  - We can lend copies of the book to library users. We can keep track of how many book copies we lend to each user. This way, we can see if we run out of a book.

Business modeling? Why?

Good modeling makes us discover how systems should work, uncover possible functionalities and bugs.

They are the basis of efficient systems that are easier to program.

This also saves money on the long run - both your client’s, and yours.

A quick note

There are many ways of modeling data. Two of the most used nowadays are:

- Relational
  - Very used in “traditional” relational databases
  - Puts data in strictly defined tables with rows and columns, and relationships between them
  - Based on relational algebra
- Object-oriented
  - Used to model object-oriented systems
  - Also used to understand real-life situations (the “business”)
  - Some databases also are object-oriented

We will learn the basics of object-oriented business modeling today. This is useful even if your application is *not* object-oriented.

How to start with data modeling?

What are you creating/updating/etc.?

You need to identify what is it that your application is working with. (Books? Reservations? Users? etc.) You will have many of these, and they are related.

We will call them business objects. These come from your requirements.

How to start with data modeling?

When you're gathering requirements, the user tells you what they need to do. At that moment, listen to:

- What nouns they are using in their words? -> these become the business objects
- What do they do with the objects? -> these become the actions

A bit about data modeling

Example for a POS (point of sale system - a checkout in a supermarket). Imagine we have a cashier user who says this:

“I scan articles that the client buys. I need the system to recognize the articles when I scan them. The articles should appear with their price on the screen.”

Photo by Jack Sparrow : https://www.pexels.com/photo/couple-buying-groceries-at-a-supermarket-4199490/

A bit about data modeling

Example for a POS (point of sale system - a checkout in a supermarket):

“I scan articles that the client buys. I need the system to recognize the articles when I scan them. The articles should appear with their price on the screen.”

What are the business objects? Let’s look at the nouns (other than the system itself and the users).

A bit about data modeling

Example for a POS (point of sale system - a checkout in a supermarket):

“I scan articles that the client buys. I need the system to recognize the articles when I scan them. The articles should appear with their price on the screen.”

In this case, we have one business object: an article.

A bit about data modeling

Let’s analyze that article. We will do that in two ways:

- What characteristics does it have?
- What do we do with it?

A bit about data modeling

Example for a POS (point of sale system - a checkout in a supermarket):

“I scan articles that the client buys. I need the system to recognize the articles when I scan them. The articles should appear with their price on the screen.”

Characteristics:
- The article has a price.

What we can do:
- We can buy the article.

Photo by Jack Sparrow : https://www.pexels.com/photo/couple-buying-groceries-at-a-supermarket-4199490/

A bit about data modeling

A bit about data modeling

Now that we know this, we could start modeling a simple system.

We know the system has to store information about articles. We can think of a CRUD for articles:
- Create articles
- Read articles
- Update articles
- Delete articles

Plus, we know we need to buy them, so:
- Buy articles

A bit about data modeling

Article
+ price
+ buy() + read() + create() + update() +delete()

Data modelling

You also want to think about the relationships between business objects:

- Some objects are a kind of other objects. (For example, a food is a type of article. Maybe we need special management for food items.)

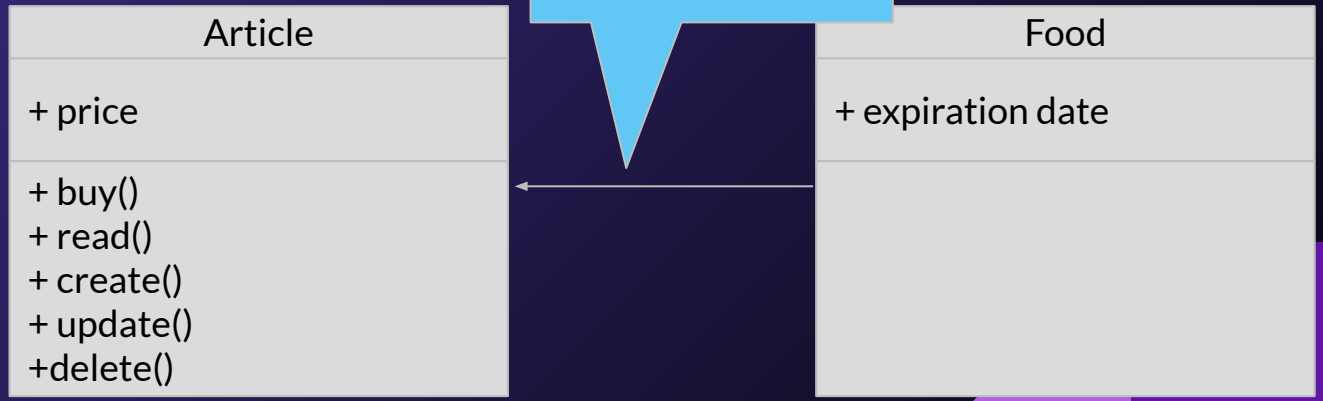
Data modelling

You also want to think about the relationships between business

- Some objects are a kind of other o ple, a food article. Maybe we need special ma od items.)

Food has everything that Article has, plus an expiration date

Inheritance. The arrow means “food is a type of article”



Data modelling

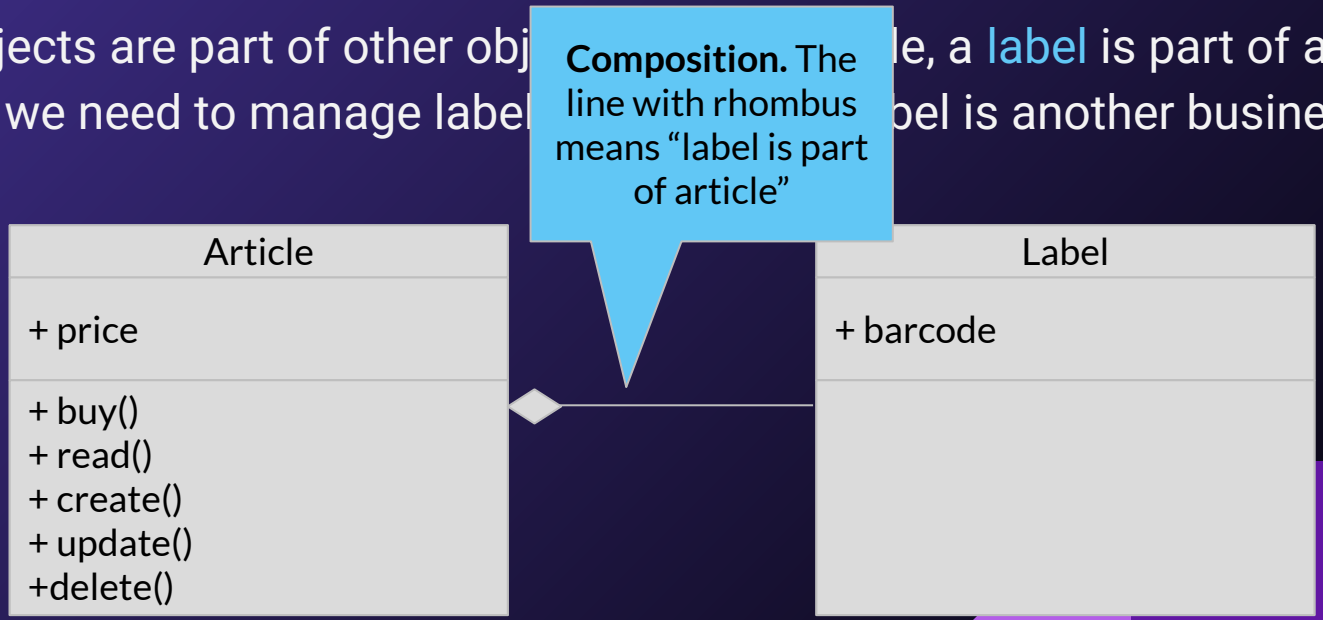
You also want to think about the relationships between business objects:

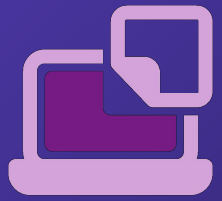
- Some objects are part of other objects. (For example, a label is part of an article. If we need to manage labels separately, a label is another business object.)

Data modelling

You also want to think about the relationships between business objects:

- Some objects are part of other obj le, a label is part of an article. If we need to manage label bel is another business object.)





Created by HideMaru
from Noun Project

Activity time: Modeling a simple system

Imagine you will create a system for a used car dealership.

Requirements: The employees want to use this system to:

- Keep track of the current car inventory
- Sell cars
- Keep track of new and existing clients

Create a model of this system.

- What business objects (nouns) are there?
  - What are their characteristics? What can you do with them?
- What are the relationships between the objects?

You have 10 minutes.

Icon by HideMaru for Noun Project

From use cases to an API

For a simple CRUD for books (library inventory):

User:

- Library manager: manages library inventory

Use cases:

- Library manager:
  - Add books to the inventory
  - Delete books to the inventory
  - Update books (eg. change the number of available copies)

From use cases to an API

Business model:

Book
+ code + name + author + edition + number of copies
+ create() + update() +delete()

From use cases to an API

API: (for frontend to do database operations into the backend)

`GET /books`
gets a list of all books

`GET /book/`*`code`*
Gets all data of a single book
example: `GET /book/2094c` to get info on book with code 2094c

`POST /book`
Creates a new book; data for the book is sent in the request

`PUT /book/`*`code`*
Updates the data of a single book
example: `PUT /book/2094c` updates data of book with code 2094c; data to be updated is sent in the request

`DELETE /book/`*`code`*
Deletes a book from the database
example: `DELETE /book/2094c` deletes this book

More on this topic

For more, you can explore:

- The Basic Object Oriented Data Model
- A guide to the UML Class Diagrams where the symbols came from